ML assignment 2: Project work

Nina Mæland Polden og Julie Elea Fjellstad, 30.10.2026

Link til applikasjon: https://millionaire-habits-ml.streamlit.app/

1: BESKRIV PROBLEMET

OMFANG / *SCOPE*

Målet er en nettside hvor brukeren skal legge inn opplysninger om seg selv, og etter det får et maskinlæring-basert estimat på om de har en profil som ligner en millionær. Her innebærer å være millinoær en netto formue større eller lik 1 million USD.

Nettsiden er lagd for alle som ønsker å forstå hvordan inntekt, sparing og tid går hånd i hånd når det kommer til å bygge opp en formue. Siden er kun laget i sammenheng med skoleprosjektet, og er dermed ment for læring. Vi synes også at det var artig å lage en nettside for å predikere om man har vaner som ligner en millionærs sparevaner. Vi vil påpeke at det ikke er ment som økonomisk rådgiving, men mer underholdning.

Andre, lignende løsninger i dag er for eksempel sparekalkulatorer med formler på nett. Manuelt kan man regne ut en fremtidig formue med sparebeløp, avkastning og antall år. Utfordringen med dette kan for eksempel være at en må anta avkastning dersom sparebeløpet står i fond/aksjer.

En klassifiseringsmodell kan lære ikke-lineære sammenhenger mellom mange variabler direkte fra data, uten at vi må spesifisere formelen selv. Dermed får vi også sannsynligheten, ikke bare ja eller nei svar. En feature importance-analyse kan også vise hvilke faktorer som er mest vektlagt.

Løsningen består av et treningsscript som renser data og trener modellen, en lagret scikit-learn-pipeline med preprosessering og modell-trening, og en Streamlit-app som laster vektene og viser prediksjoner. Dersom feature-settet skal endres i treningen, så må input-feltene i appen endres tilsvarende. Preprosesseringen ligger derfor inne i pipelinen.

Treningen på 100.000 rader tar under ett minutt på en vanlig PC, og prosjektet krever ingen GPU. Hosting er gratis via Streamlit.

METRIKKER

I valideringsrapporten ser vi at 42,5% av profilene er millionærer, så klassene er rimelig balanserte da det er omtrent like mange i hver gruppe. Vi måler likevel hvor bra modellen er på flere måter.

ROC-AUC, hovedmetrikk, måler hvor godt modellen skiller millionærer fra ikke-millionærer, uansett hvor vi setter grensen for “ja”. Passer fordi nettsiden viser en sannsynlighet. Så har vi F1-score, precision og recall for millionærene, når modellen sier “ja” ved sannsynlighet

over 50%. Accuracy, sammenlignet med en baseline som alltid gjetter majoritetsklassen (ca. 57,5%). Log-loss er tapsfunksjonen modellene optimaliserer under trening. Den må være kontinuerlig og deriverbar for at gradient descent skal virke, noe accuracy og F1 ikke er. Log-loss brukes også til early-stopping, men det er metrikkene nevnt før vi bruker for å si om modellen er god nok. Vi sammenligner AUC på treningsdata og valideringsdata. Hvis treningstallet er mye høyere har vi overtilpasning. Vi bruker dette til å velge hyperparametre som maks dybde og regularisering. Software-metrikker: responstid per prediksjon (målt under 1 sekund) og at appen starter uten feil.

Suksesskriteriet er at modellen skal ha ROC-AUC på minst 0,80 på testsettet og være klart bedre enn baseline. Nettsiden skal kunne brukes uten forklaring, men siden produktet er et læringsverktøy, finnes det ingen direkte "business metric" i kr. Det nærmeste er kanskje da brukertilfredshet og om brukerne forstår hva resultatet betyr.

2: DATA

Vi har brukt Millionaire Habits Dataset fra Kaggle: https://www.kaggle.com/datasets/datascikhan/millionaire-habits-dataset. Datasettet har 100.000 rader og 274 kolonner fordelt på forskjellige kategorier som blant annet: demografi, karriere, utdanning, sparing, formue, investering, gjeld, produktivitet, livsstil. Labelen millionaire_status er 1 dersom netto formue er minst 1 000 000 USD. Vi har satt 80.000 til trening og 20.000 til test. Vi ser at alle kolonner som ikke lekker er 177.

Datasettet er syntetisk, så det er ikke basert på reell data, men er generert av et Python skript. Det representerer altså ingen reelle personer.

3: MODELLERING

Vi har sammenlignet en baseline med logistisk regresjon, random forest og LightGBM. LightGBM på alder, sparerate og inntekt ble best med ROC-AUC på 0,882 på testsettet. Vi implementerte også logistisk regresjon selv med gradient descent og sjekket hvilke features som betyr noe med ulike feature-sett og L1-regularisering. All kode ligger i notebooken og kan sjekkes der.

Som baseline brukte vi DummyClassifier som gjetter den vanligste klassen som er "ikke millionær". Accuracy her ble 57,5%, men AUC ble 0,50, trolig fordi den aldri vil gi "millionær".

Vi har altså logistisk regresjon som trekker en rett grense mellom gruppene, så har vi random forest som lager mange beslutningstrær på tilfeldige utvalg av dataene for så å ta snittet av dem, og så har vi LightGBM som bygger trær etter hverandre, der hvert nytt tre retter opp feilene til forrige. Alle er trent på alder, inntekt og sparing og testet på testsettet.

Modell	ROC-AUC	F1	Precision	Recall	Accuracy

Baseline	0,500	0,000	-	0,000	0,575
Logistisk regresjon	0,872	0,743	0,781	0,708	0,791
Random forest	0,860	0,739	0,750	0,726	0,781
LightGBm	0,882	0,761	0,773	0,751	0,800

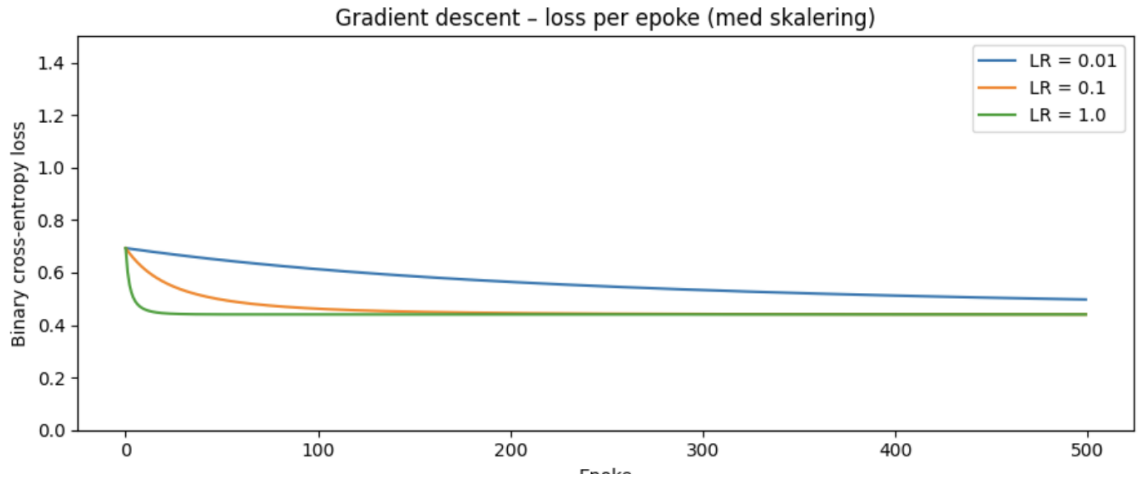
Ser at LightGBM er best på alle metrikker utenom precision, mens logistisk regresjon er nesten like god. Så sammenhengen i dataen er: høyere alder, inntekt og sparing gir høyere sjanse. Random forest er nok dårligst, trolig fordi hvert tre med standardinnstillingene vokser til det passer treningsdatene nesten perfekt, så modellen overtilpasser.

Vi skrev logistisk regresjon i NumPy med hjelp av AI, med log-loss som tapsfunksjon. Vektene starter på 0, og i hver epoke regnes gradienten ut og tar et steg i retningen som gir lavere loss.

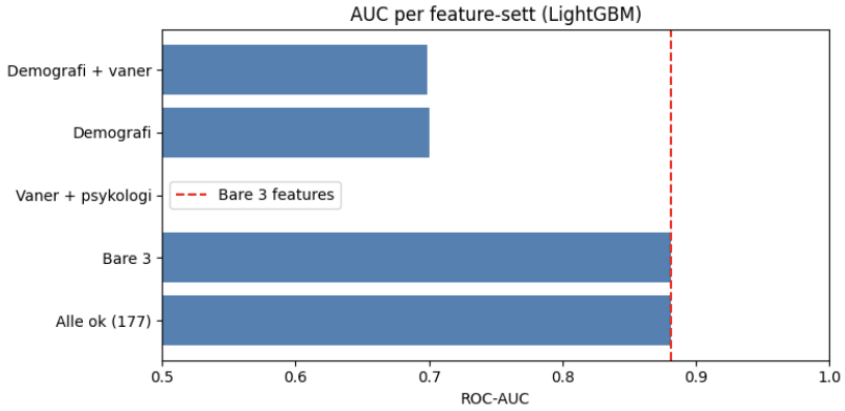
Learning rate	Loss etter 500 epoker	Vekter (alder, inntekt, sparerate)
0,01	0,497	0,376 * 0,713 * 0,444
0,1	0,441	0,786* 1,690 * 0,887
1,0	0,4406	0,811 * 1,768 * 0,913
Scikit-learn(fasit)	-	0,811*1,768 * 0,913

Med learning rate 1,0 fikk vi samme vektene som scikit-learns LogisticRegression, så implementasjonen er riktig. Med 0,01 ble stegene så små at modellen ikke er ferdig etter 500 epoker. Inntekt får størst vekt, så den betyr da mest av de tre vektene.

Uten skalering så økte loss fra 0,693 til 19,9 etter 50 epoker. Inntekt er i hundretuseneer og alder er under 100. Gradienten for inntekt blir da mye større enn de andre, og da blir stegene igjen for store.



For å finne ut hvilke features som betyr noe trente vi LightGBM på fem ulike feature-sett.



"Bare tre" er alder, inntekt og sparerate, med høy AUC på 0,8817. De tre featuresene gir ca like høy AUC som alle 177 kolonnene som ikke lekker. Vaner og psykologi gir 0,497, altså ikke bedre enn å gjette. Så ingen av vanene sier noe om hvem som er millionær i det datasettet her.

L1-regularisering bekrefter dette, vi trente logistisk regresjon med L1-straff på alle 177 kolonnene. L1 setter vektene til uviktige features til 0, og jo lavere C, jo strengere straff.

C	Features med vekt != 0	AUC
10	175	0,877
1	171	0,877
0,1	144	0,878
0,01	13	0,880
0,001	6	0,870

Med C = 0,01 klarer modellen seg med 13 features og får litt høyere AUC enn med alle. Ved C = 0,001 overlever bare seks: income_percentile, savings_rate, years_of_experience, annual_salary, total_annual_income og age. Alle handler om inntekt, sparing og alder, ingen om vaner. Det er grunnen til at vi bare tok med spørsmål om tre features på nettsiden.

Vi endte opp med å velge LigthGBM fordi den hadde høyest AUC og bare trenger tre ting fra brukeren. Siden AUC var på treningsssettet: 0,894 og testsettet: 0,882, er forskjellen så liten at modellen ikke har pugget treningsdataene.

Forvirringsmatrisen viser hvilke feil den valgte modellen gjør på testsettet. Feilene er omtrent like fordelt.

	"Ikke millionær"	"Millionær"
Faktisk ikke millionær	9621	1881
Faktisk millionær	2136	6362

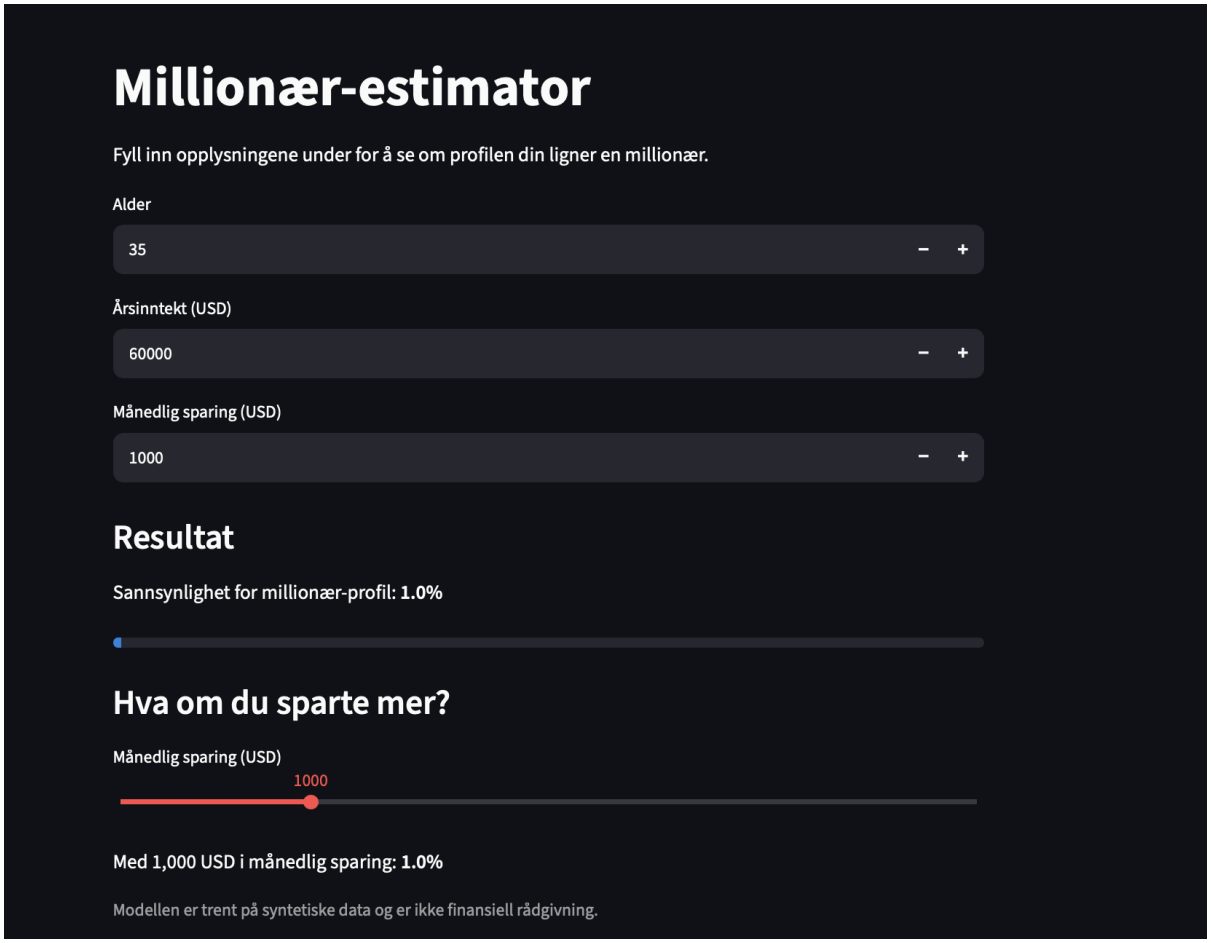
Feature-analysen viser at flere vaner ikke vil hjelpe, mulige forbedringer kan derfro være å heller justere grensen for å si "ja" hvis den ene feiltypen er verre enn den andre. Grensen for

å si ja nå er 50%. En kunne også sett nærmere på personene modellen tar feil på, for eksempel om de ligger nær grensen på 1 million.

4: DEPLOYMENT

Modellen kjører i en Streamlit-app: https://millionaire-habits-ml.streamlit.app/. Brukeren fyller bare inn alder, inntekt (per år) og hvor mye de sparer per mnd. Så regner appen ut sparerate, sender input gjennom pipelinen og viser deretter sannsynligheten for at profilen tilhører en millionær.

Treningsskriptet lagrer hele pipelinen med joblib. Appen (app.py) laster den en gang ved oppstart med @st.cache_resource. Avhengigheter med låste versjoner ligger i requirements.txt sånn at alle kan kjøre appen lokalt med streamlit run app.py eller deploye den på nytt. Appen sjekker at det brukeren skriver inn er fornuftig med tanke på alder og månedlig sparing ut fra hva inntekt er. For eksempel kan du ikke spare mer i måneden enn det du tjener i måneden.



5: REFERANSER

1. Shair Khan. *Millionaire Habits Dataset – Wealth, Lifestyle, Productivity & Financial Behavior* (v1.0.0). Kaggle, 2026. https://www.kaggle.com/datasets/datascikhan/millionaire-habits-dataset/data

2. Forelesninger DAT158, HVL, høsten 2026.

3. Bruk av AI-verktøy: Claude (Anthropic) er brukt til å vurdere datasettet og identifisere leakage, gå over rapportstruktur og rette opp i feil, samt. skrive kode. All kode og tekst er gjennomgått og tilpasset.